



Git Conventions Guide – Commits & Branches

This document defines the Git conventions to be followed throughout the project. The goal is to ensure:

- Clear and readable Git history
- Strong traceability of individual contributions
- Efficient code reviews
- Easy project handover to another development team



Objectives

- Quickly identify the **nature of commits**
- Clearly link code changes to **GitHub Issues**
- Facilitate **individual and collective evaluation**
- Fully leverage **GitHub tools** (issues, projects, releases)



Commit Nature (Emojis)

Each commit **must start with an emoji** to visually indicate its intent.

Emoji	Description
✨	Introducing new features
🧠	Improving code structure / formatting
⚡	Improving performance
💧	Removing code or files
🐛	Fixing a bug
🚚	Critical hotfix
📄	Writing documentation
🚀	Updating UI / styles
🔒	Fixing security issues
✅	Releases / version tags
🚧	Work in progress
❤️	Fixing CI build
⬇️	Downgrading dependencies
⬆️	Upgrading dependencies
👷	Adding CI/CD pipeline
📊	Adding analytics

Emoji	Description
♻️	Refactoring code
—	Removing a dependency
+	Adding a dependency
🔑	Changing configuration
🌐	Internationalization / localization
⏮️	Reverting changes
🗉	Applying code review feedback
🏗️	Architectural changes
🔧	Tests

🔖 Commit Message Rules

A good commit message must:

- Be **short** (72 characters or less)
- Use the **present tense**
- Use the **imperative mood**
- Clearly describe the **affected scope**
- Be linked to a **GitHub Issue**

📌 Mandatory Commit Format

`feat(scope) :emoji: clear and concise message`

☑️ Recommended Commit Examples

feat(views) ✨ add start simulation button feat(controllers) ✨ add network routes controller fix(map) 🐛 fix marker zoom issue docs(readme) 📖 add project documentation refactor(api) ♻️ simplify simulation service test(auth) 🔧 add login unit tests

❌ Not Recommended Commit Messages

add button fix bug update code

❌ These messages lack scope, intent, and traceability.

🌿 Branch Naming Convention

🎯 Objectives

- Clearly identify **what is being worked on**
- Link code changes to **GitHub Issues**
- Simplify tracking, reviews, and evaluation

Branch Naming Format

The general format for all branches is:

`TaskType/FeatureName/GH-{IssueNumber}-Description` The general format for all branches is:

`TaskType/FeatureName/GH-{IssueNumber}-Description` The general format for all branches is:

`TaskType/FeatureName/GH-{IssueNumber}-Description` The general format for all branches is:

`TaskType/FeatureName/GH-{IssueNumber}-Description` The general format for all branches is:

`TaskType/FeatureName/GH-{IssueNumber}-Description`

Branch Name Breakdown

1. TaskType

Indicates the type of work:

- `Feature`
- `Bug`
- `Refactoring`
- `HotFix`
- `Docs`

2. FeatureName

Functional domain or module:

- `auth`
- `map`
- `simulation`
- `api`
- `frontend`
- `backend`
- `docs`

3. Ticket ID

The associated GitHub Issue number (**mandatory**)

Branch Examples

`Feature/Auth/GH-12-AddLogin` `Bug/Map/GH-21-FixZoomIssue` `Refactoring/API/GH-18-CleanSimulationService` `Docs/Architecture/GH-05-AddSystemOverview` `HotFix/Backend/GH-30-FixProductionCrash`

GitHub Best Practices

- **1 issue = 1 branch**
- Commits must be made **only** on the related branch

- All changes must go through a **Pull Request**
- Use GitHub features exclusively:
 - Issues
 - Projects / Boards
 - Labels
 - Pull Requests
 - Releases & Tags
- No external project management tools (Trello, Notion, etc.)

Releases & Deliverables

Project deliveries are materialized using **Git tags**:

- **v0.1.0** → First intermediate delivery (01/02/2026)
- **v0.2.0** → Second intermediate delivery (01/03/2026)
- **v1.0.0** → Final delivery (29/03/2026)

Each release must:

- Be tagged
- Be associated with closed issues
- Contain stable and reviewable code

Documentation as Code

Documentation is produced directly in the repository:

- README files
- GitHub Wiki
- **docs/** directory
- Diagrams (Mermaid, PlantUML, etc.)

The goal is to ensure that another development team can easily understand, maintain, or extend the project without external knowledge.